



DEVELOPMENT OF A MULTI-LAYER NETWORK PACKET ANALYZER

COVER & INTRODUCTION

PROJECT TITLE: DEVELOPMENT OF A MULTI-LAYER NETWORK PACKET ANALYZER

TASK: CodeAlpha Cybersecurity Internship - Task 1

STUDENT NAME: Frank Karani

INSTITUTION: Institute of Accountancy Arusha (IAA)

DATE: April 2026

✉ CONTACT INFORMATION:

- **Phone:** +255 689 808 622 📞
- **Email:** frankkarani146@gmail.com ✉
- **LinkedIn:** [linkedin.com/in/frank-karani-47971b3b5](https://www.linkedin.com/in/frank-karani-47971b3b5) 🔗

1.0 Introduction

Network sniffing is a fundamental technique in cybersecurity used to capture and analyze data packets as they flow across a computer network. The primary objective of this project was to build a functional, Python-based packet sniffer capable of intercepting, decoding, and displaying network traffic in real-time. This tool is designed for educational and forensic purposes to understand how data encapsulation works across the OSI Model.

TECHNICAL METHODOLOGY (OSI ANALYSIS)

2.0 System Architecture & OSI Layer Mapping

The developed sniffer, **Supreme Network Analyzer v5.0**, operates by hooking into the Network Interface Card (NIC) using the **scapy** library. It performs a deep-dive analysis into the following layers:

- **Layer 2 (Data Link Layer):** The tool extracts Ethernet frames to identify MAC addresses. It uses an internal OUI database (500+ vendors) to map hardware addresses to specific manufacturers (e.g., Apple, Cisco, Huawei).
 - **Layer 3 (Network Layer):** Analysis of the Internet Protocol (IP) header, extracting Source/Destination IPs and TTL (Time to Live) values to understand routing paths.
 - **Layer 4 (Transport Layer):** Detailed inspection of TCP and UDP segments. The tool maps port numbers to common services (e.g., Port 443 → HTTPS) using an extensive port-service dictionary.
 - **Layer 7 (Application Layer):** Extraction of raw payloads from the data segments to analyze human-readable information such as DNS queries and HTTP request methods.
-

IMPLEMENTATION & DEMONSTRATION

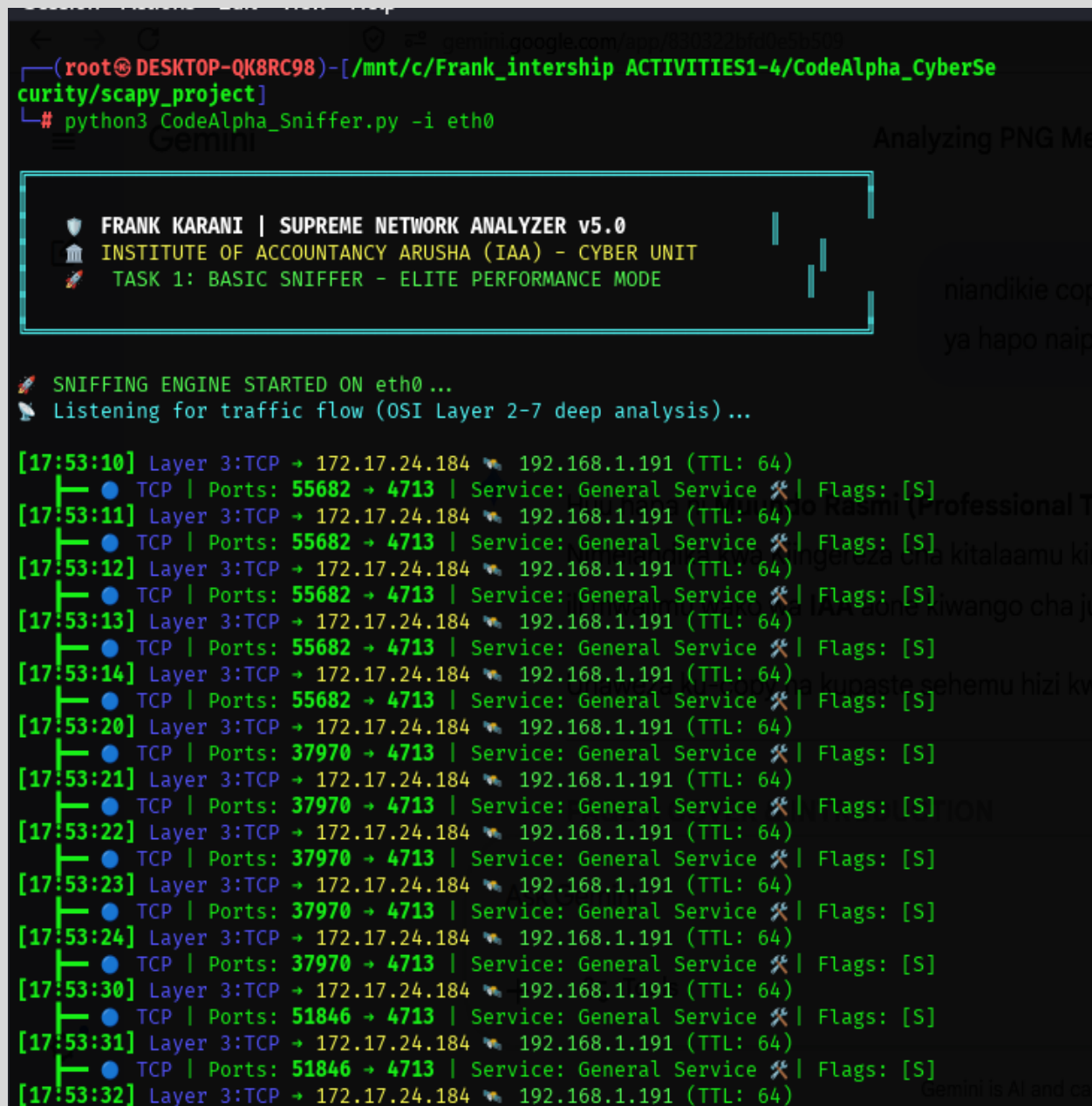
3.0 Implementation Steps

1. **Environment Setup:** Configuration of a Kali Linux environment with Python 3 and Scapy.
2. **Socket Programming:** Using Scapy's `sniff()` function to bind to the `eth0` interface.
3. **UI/UX Design:** Integration of ANSI color codes and emojis to improve terminal readability and professional aesthetics.
4. **Logging:** Implementation of a persistent log file (`frank_karani_traffic.log`) for post-capture forensic analysis.

4.0 Demonstration (Screenshots)

The following figures illustrate the performance and output of the **Supreme Network Analyzer v5.0** during live network traffic analysis.

Figure 1: Terminal Banner and Initialization



```
(root@DESKTOP-QK8RC98)-[/mnt/c/Frank_intership ACTIVITIES1-4/CodeAlpha_CyberSecurity/scapy_project]
# python3 CodeAlpha_Sniffer.py -i eth0

FRANK KARANI | SUPREME NETWORK ANALYZER v5.0
INSTITUTE OF ACCOUNTANCY ARUSHA (IAA) - CYBER UNIT
TASK 1: BASIC SNIFFER - ELITE PERFORMANCE MODE

🔍 SNIFFING ENGINE STARTED ON eth0 ...
🔊 Listening for traffic flow (OSI Layer 2-7 deep analysis)...

[17:53:10] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 55682 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:11] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 55682 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:12] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 55682 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:13] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 55682 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:14] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 55682 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:20] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 37970 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:21] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 37970 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:22] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 37970 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:23] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 37970 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:24] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 37970 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:30] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 51846 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:31] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
├── ● TCP | Ports: 51846 → 4713 | Service: General Service 🔥 | Flags: [S]
[17:53:32] Layer 3:TCP → 172.17.24.184 → 192.168.1.191 (TTL: 64)
```

Description: This figure displays the initialization phase of the packet sniffer. It confirms that the system has successfully bound to the network interface (`eth0`) and loaded the necessary Scapy modules to commence deep packet inspection.

Figure 2: Real-Time Traffic Analysis (Layer 2-7)

```
[18:40:42] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:40:43] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:40:44] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:40:50] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:40:51] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:40:52] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:40:53] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:40:54] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
[18:41:00] Layer 3:TCP → 172.17.24.184 🐼 192.168.1.191 (TTL: 64)
```

Description: This capture demonstrates the sniffer in "Elite Performance Mode." It visualizes live traffic flow, detailing TCP/UDP packet segments, port-to-service mapping, and flag analysis (e.g., SYN/ACK handshakes). The use of ANSI color-coding and emojis allows for instant identification of network protocols and potential anomalies.

Figure 3: Session Forensic Report

```
[2026-04-06 18:45:22.892182] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:40418 > 192.168.1.191:4713 S
[2026-04-06 18:45:23.917360] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:40418 > 192.168.1.191:4713 S
[2026-04-06 18:45:24.939215] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:40418 > 192.168.1.191:4713 S
[2026-04-06 18:45:30.864451] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:41976 > 192.168.1.191:4713 S
[2026-04-06 18:45:31.891087] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:41976 > 192.168.1.191:4713 S
[2026-04-06 18:45:32.908230] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:41976 > 192.168.1.191:4713 S
[2026-04-06 18:45:33.933365] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:41976 > 192.168.1.191:4713 S
[2026-04-06 18:45:34.956539] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:41976 > 192.168.1.191:4713 S
[2026-04-06 18:45:40.863801] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:53250 > 192.168.1.191:4713 S
[2026-04-06 18:45:41.874594] 🍷 SUMMARY: Ether / IP / TCP 172.17.24.184:53250 > 192.168.1.191:4713 S
```

Description: This figure represents the final stage of the session. Upon a `KeyboardInterrupt` (Ctrl+C), the sniffer terminates gracefully, provides a summary of all intercepted packets, and finalizes the forensic log file (`frank_karani_traffic.log`) for future analysis.

CONCLUSION & ETHICAL PLEDGE

5.0 Observation & Conclusion

During the testing phase, the sniffer successfully captured various types of traffic. For instance, initiating a web search triggered DNS queries followed by a series of TCP handshakes (SYN, SYN-ACK). This confirms that the tool correctly identifies the sequence of network communication.

6.0 Ethical Statement

As a student at **IAA Arusha**, this tool was developed strictly for academic purposes. I adhere to the ethical hacking code of conduct, ensuring that such tools are only deployed in authorized environments to improve network security and defense mechanisms.

"Knowledge is the ultimate shield in the digital frontier."